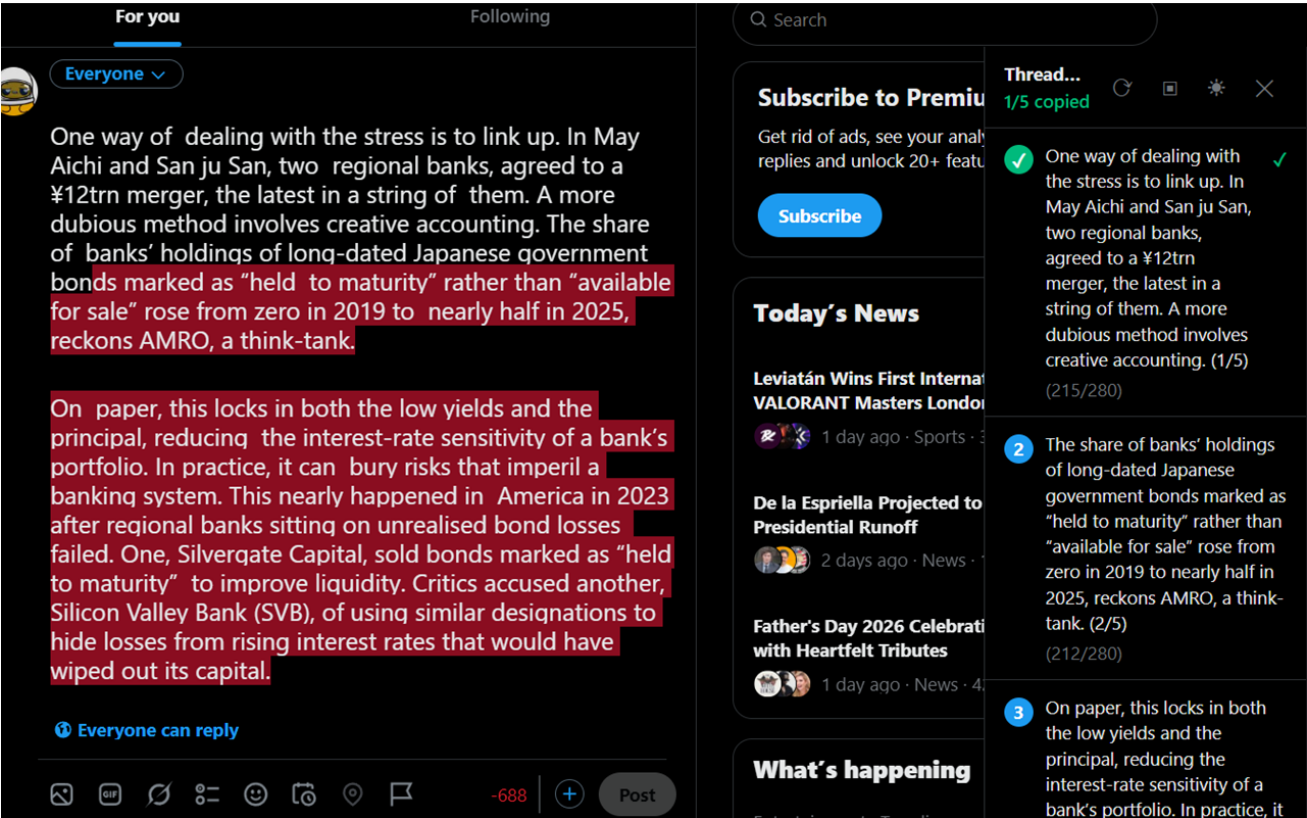


X-Split: PM Case Study

X-Split is a lightweight browser extension that helps free-tier X users turn long drafts into readable thread-sized posts without leaving the compose flow. I built it as a focused utility rather than a broader social media tool, with the goal of removing one repetitive pain point instead of recreating an entire writing platform.

23	5	June 29, 2026
Chrome installs	Active users	Public launch



Extension interface shown inside the X compose flow. The product stays in-context instead of pushing users into a separate writing tool.

Quick Facts

Product	Browser extension for splitting long X posts into thread-sized chunks
Role	Solo builder covering product, implementation, launch, and early distribution
Launch date	June 29, 2026

Platforms	Chrome live, Firefox build prepared
Traction as of July 14, 2026	23 installs, 5 active users
Main acquisition channels	Reddit, Product Hunt, X
Audience signal	Users concentrated in the United States, with many on ChromeOS

Problem

Free-tier X users are limited to 280 characters per post. For anyone who writes long-form content, that creates a manual workflow: split the draft, preserve readability, add thread numbering, and move the pieces back into X one by one.

That workflow is frustrating enough to matter but small enough that most existing products solve it indirectly. Tools like Typefully or Hypefury offer broader writing, scheduling, and growth workflows, but they also introduce more setup, more context switching, and more product than some users need.

The gap I wanted to explore was simple: could a narrow tool embedded directly in X remove that friction more effectively than a more powerful external product?

Why This Was Worth Building

I did not approach X-Split as a generic side project. I approached it as a narrow product bet. The hypothesis was that some users do not want a larger writing platform or a paid scheduling tool. They want a fast way to turn one long thought into a readable thread without leaving the place they are already writing.

That made the opportunity worth testing. The problem was clear, the alternatives were easy to study, and the solution space had room for a lower-friction option. Even if the product stayed small, it was a good test of whether a focused utility could win on convenience rather than feature breadth.

My Role

I owned the project end to end as a solo builder. That included defining the problem, identifying the target user, writing the PRD, shaping the scope, making the key tradeoff decisions, shipping the extension, preparing the listing, and handling the initial launch and distribution myself.

Because there was no larger team, this was not a traditional stakeholder-heavy PM project. The product management work showed up in problem selection, positioning, scope control, sequencing, and launch ownership. I treated distribution as part of the product job because without public launch effort, I would have learned very little about whether the product resonated beyond my own assumption.

Research and Validation

The initial validation for X-Split was lightweight and founder-led rather than formal. I started from a visible workflow pain, documented it in a PRD, wrote user personas, and compared the product against adjacent tools to understand where the real differentiation might be.

The clearest insight from that work was that most alternatives treat thread writing as a separate workflow in a separate app. X-Split was intentionally positioned around the opposite idea: keep the user inside X, reduce setup, and solve only the immediate job of splitting a long post cleanly.

Post-launch, I used public distribution channels to test whether the product could attract users who had no relationship to me. That is still a small sample, but it gave me an early signal that the problem was real enough to produce installs from public channels rather than from private sharing alone.

Product Strategy and Scope

The core strategy was to compete on friction, not feature depth. I did not try to match full social media writing platforms. Instead, I focused on a narrower promise: help free-tier X users turn one long draft into a readable thread inside the existing compose workflow.

That strategy drove the v1 scope. I included sentence-aware splitting, URL-aware character counting, automatic numbering, inline preview, re-splitting after edits, and lightweight settings. I deliberately left out scheduling, analytics dashboards, account linking, draft management, templates, and posting automation.

That discipline mattered. If the product had expanded too early, it would have lost its clearest advantage: solving one annoying task with very little setup or learning curve.

What I Built

I built X-Split as a browser extension that watches for long-form input inside the X compose experience, splits the text into thread-sized chunks, and shows those chunks in a sidebar for review and copying.

The first version focused on usability over breadth:

- Sentence-aware splitting so posts read naturally
- URL-aware counting based on X link rules
- Automatic thread numbering
- Inline preview of all chunks
- Re-splitting after edits
- Fill mode and formatting options for users who want tighter control

The product stays intentionally simple. It does not try to post on the user's behalf or become a full content workflow tool.

Launch and Early Results

I launched X-Split publicly on June 29, 2026. Because I knew passive launch would not be enough for a niche extension, I treated go-to-market as part of the product work and distributed it through [r/chromeextension](#), [r/sideprojects](#), Product Hunt, and X.

Reddit drove most of the day-one traffic. Product Hunt added some visibility around launch, while X was a weaker channel because I did not have much existing reach there.

As of July 14, 2026, the Chrome Web Store dashboard showed:

-
- 23 installs
 - 5 active users
 - Users concentrated in the United States
 - A notable share of users on ChromeOS

These are early numbers, not breakout traction. What matters is that the product reached real users through public channels and produced a usable first layer of market signal.

What I Learned

The biggest lesson from X-Split was that narrow utility products depend on clarity more than complexity. The product did not need a long feature list to be useful. It needed a sharp problem definition, a simple workflow, and positioning that made the benefit obvious within seconds.

I also learned that launch is inseparable from validation at this scale. Without active distribution, I would have learned very little. Even modest traction helped answer an important question: would anyone outside my own circle care enough to install this?

Finally, I learned the limits of lightweight validation. I now have enough signal to believe the problem is real, but not enough to claim strong retention or product pull. The next phase would require more structured feedback, stronger activation insight, and better understanding of what separates installers from repeat users.

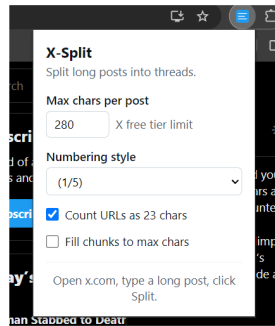
What I Would Do Next

If I continued developing X-Split, my next priority would be learning rather than expansion. The main questions now are about activation and repeat usage: how many installers actually use the extension to create a thread, how often active users return, and where the workflow breaks down for people who try it once and do not come back.

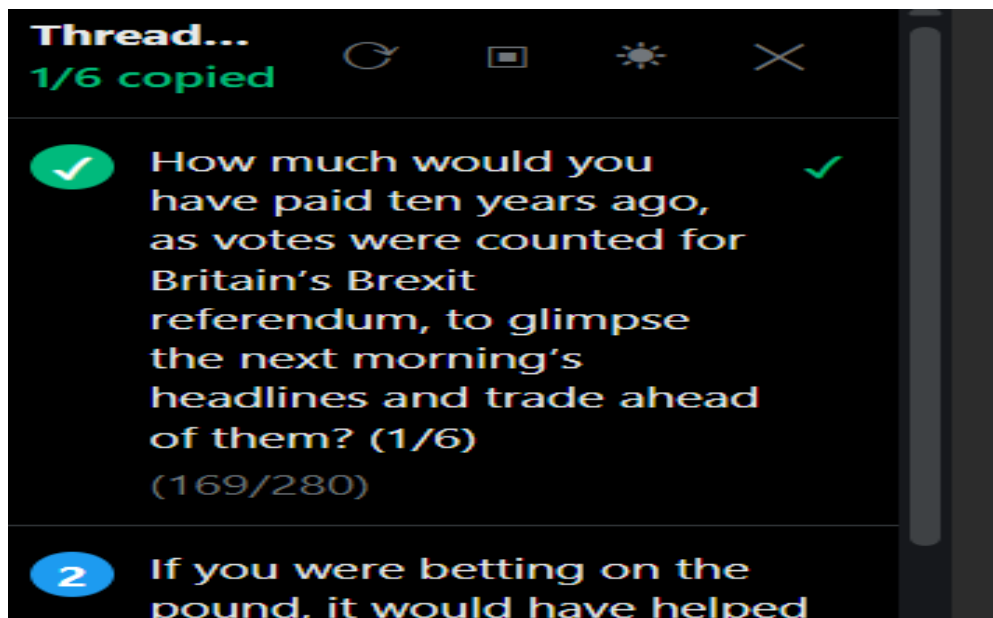
I would focus first on the smallest changes that strengthen the core use case, such as onboarding, messaging clarity, and workflow polish. Only after that would I revisit broader feature ideas. The strategic question is whether X-Split should remain a narrow utility or grow into a larger thread-writing product. My instinct would be to protect the narrow utility advantage until user behavior clearly justifies expanding beyond it.

Product Artifacts

The shipped extension included a collapsible in-page thread panel, a compact settings surface, and a simple visual identity designed for store readiness. I am including these screens in the case study because they show the actual product decisions made in v1, not speculative concepts or mockups.



Settings popup from the store assets. This shows the lightweight configuration surface shipped with v1.



Close-up of settings of the side panel.